



ChopChop: A Programmable Framework for Semantically Constraining the Output of Language Models

SHAAN NAGY*, University of California San Diego, USA
TIMOTHY ZHOU*, University of California San Diego, USA
NADIA POLIKARPOVA, University of California San Diego, USA
LORIS D'ANTONI, University of California San Diego, USA

Language models (LMs) can generate code but cannot guarantee its correctness—often producing outputs that violate type safety, program invariants, or other semantic properties. Constrained decoding offers a solution by restricting generation to only produce programs that satisfy user-defined properties. However, existing methods are either limited to syntactic constraints or rely on brittle, ad hoc encodings of semantic properties over token sequences rather than program structure.

We present ChopChop, the first programmable framework for constraining the output of LMs with respect to semantic properties. ChopChop introduces a principled way to construct constrained decoders based on analyzing the space of programs a prefix represents. It formulates this analysis as a realizability problem which is solved via coinduction, connecting token-level generation with structural reasoning over programs. We demonstrate ChopChop's generality by using it to enforce (1) equivalence to a reference program and (2) type safety. Across a range of models and tasks, ChopChop improves success rates while maintaining practical decoding latency.

CCS Concepts: • **Software and its engineering → General programming languages**; • **Computing methodologies → Machine learning**; • **Theory of computation → Semantics and reasoning**.

Additional Key Words and Phrases: LLM, Semantic Constrained Decoding, Coinduction

ACM Reference Format:
Shaan Nagy, Timothy Zhou, Nadia Polikarpova, and Loris D'Antoni. 2026. ChopChop: A Programmable Framework for Semantically Constraining the Output of Language Models. *Proc. ACM Program. Lang.* 10, POPL, Article 66 (January 2026), 28 pages. https://doi.org/10.1145/3776708

1 Introduction

Language models (LMs) have fundamentally transformed how we interact with code—generating functions, completing boilerplate, and even suggesting entire programs. Yet despite their success, LMs offer no guarantees of correctness: they often produce code that looks plausible but violates critical syntactic or semantic properties. This limitation arises because LMs generate text in a purely probabilistic fashion—at each step, they sample the next token from a distribution conditioned on the sequence they have produced so far.

Constrained decoding addresses this problem by enforcing constraints on the sequence an LM generates [1, 14, 31, 42, 45]: a constrained decoder modifies the sampling process to forbid selecting next tokens that would violate the constraint. Concretely, a constrained decoder performs a

*Shaan Nagy and Timothy Zhou contributed equally to this paper.

Authors' Contact Information: Shaan Nagy, University of California San Diego, USA; Timothy Zhou, University of California San Diego, USA; Nadia Polikarpova, University of California San Diego, USA; Loris D'Antoni, University of California San Diego, USA.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2475-1421/2026/1-ART66
https://doi.org/10.1145/3776708

completability analysis at each step to determine whether the partially generated sequence can still be extended into a completion that satisfies the constraints. If not, the LM backtracks and tries another token. By construction, any output a constrained decoding algorithm returns satisfies the user-provided constraints.

Existing constrained-decoding techniques are limited in either *expressivity* (the complexity of the constraints they can enforce) or *programmability* (the ease with which users can specify constraints).

Programmable but not Expressive. Most of the constrained-decoding literature is interested in the problem of forcing the LM to only produce sequences that are accepted by a user-given context-free grammar (CFG) [30, 35, 36, 41]. While CFGs provide a programmable interface for specifying *syntactic* constraints, many desirable *semantic* properties—such as type safety or adherence to invariants—cannot be expressed as context-free languages.

Expressive but not Programmable. Agrawal et al. [1] proposed an automaton-based framework (which they call monitors) to connect string-based static analyses (e.g., those offered by a Language Server Protocol) to a constrained decoder. Mündler et al. [31] implement a monitor to enforce well-typedness of TypeScript code. Although monitors are expressive, they operate on the string representation of a program. Because semantic constraints are usually defined over abstract syntax trees, it is nontrivial (and often unnatural) to translate semantic constraints into a corresponding completability analysis over strings. For example, while an expression like `1 + myString` would likely be considered ill-typed as a *complete* program, as an *incomplete* program it may still admit a valid continuation (e.g., `1 + myString.length`). Furthermore, because monitors are not defined over abstract syntax, they must manually handle syntax-level concerns—lexing, precedence, left recursion, and so on—failing to separate syntactic and semantic constraints. By operating over concrete syntax, monitors sacrifice *programmability* and offer no clear path for enforcing program equivalence or other complex semantic invariants.

In this paper, we ask:

*Can we design a framework for constrained decoding that is both **expressive** and **programmable**?*

Our approach. We present ChopChop, a framework that enables programmable constrained decoding. The key novelty of ChopChop is that it allows one to constrain the abstract syntax trees corresponding to the output sequences produced by an LM.

Conceptually, given a semantic constraint over abstract syntax trees, ChopChop has to decide whether a sequence of tokens in the concrete syntax (i.e., the output of an LM) can be completed so that parsing it will produce an abstract syntax tree that satisfies the given constraint. Given a sequence of tokens, ChopChop solves the above problem in two steps: (*i*) it computes a representation of the set of ASTs (a *program space*) producible by completing the token sequence with some suffix string—and (*ii*) it checks whether any AST in this space satisfies the semantic constraint.

More concretely, to use ChopChop, users provide two inputs: (*i*) a *grammar* defining the concrete syntax of the language and how abstract syntax trees can be extracted from sequences in the concrete syntax, and (*ii*) a set of *semantic pruners* which describe how to prune constraint-violating ASTs from a program space. Decoupling the semantic constraint from concrete program syntax separates concerns, allowing for an interface that is easy to program.

The key programming-language insight that enables ChopChop's pipeline to work despite having to deal with infinite sets of ASTs is the following: program spaces can be represented as *regular codata* and semantic pruners can be implemented via *corecursion* in the style of [20].

Applications. We demonstrate the generality of ChopChop through two diverse applications:

- **Program equivalence-guided decoding:** We constrain a language model to generate programs equivalent, modulo term rewriting, to a reference program. Here, the semantic pruner uses e-graphs [46] as program spaces to efficiently represent all programs that are equivalent to the reference program.
- **Type-safe decoding:** We constrain a language model to emit only well-typed programs in a subset of TypeScript. Here, the semantic pruner lifts type checking to operate over co-inductively defined sets of abstract syntax trees—i.e., the pruner constrains away possible program completions that will not typecheck.

For these domains, ChopChop produces programs that satisfy given semantic constraints more often than both unconstrained language models and approaches that only constrain the program syntax.

Contributions.

- The first *programmable framework for semantic constrained decoding* (Section 2).
- A *formal connection between semantic constrained decoding and realizability* in program synthesis, showing that enforcing semantic properties during generation reduces to checking whether the set of possible completions contains a valid program (Section 3).
- A *novel constrained decoding algorithm based on corecursion*; the algorithm incrementally builds and analyzes a coinductive representation of the space of programs obtainable from a prefix (Section 4).
- An *implementation of our framework* in a tool, ChopChop, which we instantiate in two challenging domains—equivalence-guided decoding and type-safe generation—demonstrating the generality of our approach (Section 5).
- An *evaluation* of ChopChop on the two domains—ChopChop produces programs that satisfy given semantic constraints more often than both unconstrained and purely syntax-constrained baselines while incurring a small computational overhead (Section 6).

Section 7 discusses related work, and Section 8 concludes. An extended version of this paper is available on arXiv [33] and includes appendices containing proofs and supplemental material.

2 Overview of ChopChop

We illustrate our approach with the following simple prompt one might task an LM with:

Generate a sum of odd integers whose total is even.

For example, the expression 5+7 is a valid solution, since 12 is even and both 5 and 7 are odd. In general, there is no *guarantee* that an LM will produce a valid response—it could generate a sum with even numbers (e.g., 2+2), an odd total (e.g., 1+1+1), or totally nonsensical output (e.g., banana).

A user of ChopChop can enforce the constraints mentioned in the prompt by providing two inputs:

(1) A *grammar* (Figure 1) defining the syntax of the language, annotated with rules for translating parse trees to ASTs.
(2) A set of *semantic pruners* (Figures 3a and 3b) each representing a constraint on ASTs. At a high-level, a pruner is a function that takes in a representation of a space of programs and returns (an approximation of) the subspace of constraint-satisfying programs.

```
INT  ::=  [0-9]+
  E  ::=  INT            {Lit $1}
      |   E "+" INT      {Sum $1 $3}
```

Fig. 1. A left-recursive grammar for the language of integer sums. Right-hand annotations specify parse tree-to-AST translation (e.g., {Sum $1 $3} applies the Sum constructor to the first and third arguments of the second production).

Given these inputs, ChopChop guarantees that any generated program is syntactically valid and satisfies the provided semantic constraints—in our case, the output being a sum of odd integers with an even total.[1] Crucially, ChopChop enforces constraints *during generation*, rather than waiting for the LM to produce a complete program and then retrying from scratch upon failure.

2.1 Semantic Constrained Decoding as Realizability

Returning to our example, suppose the LM has already generated the prefix 1+2, and the LM's top two choices for the next token are + and 2. ChopChop disallows + since any program starting with 1+2+ will contain an even integer. However, it allows the next token 2 as it can still lead to a valid completion (for example, 1+221).

To rule out the token +, ChopChop must determine that no completion of the sequence 1+2+ parses to an AST satisfying our example constraints. We say that ChopChop solves *semantic constrained decoding* because it prunes the LM's choices based on semantic constraints over ASTs (as opposed to *syntactic constrained decoding* which enforces shallow syntactic properties of the token stream). It is challenging to prune the LM's choices directly because constraints are expressed at the *AST* level, while 1+2+ is an incomplete *string* prefix.

Our first key insight is that **a prefix represents the space of programs corresponding to its completions.** Instead of attempting to reason about AST-level properties on *string* completions of 1+2, ChopChop reasons about the *space of ASTs* that this prefix represents. In the synthesis literature, checking satisfiability of constraints over spaces of ASTs is studied under the name *realizability* [18]. The next section outlines our realizability-based approach to semantic constrained decoding in more detail.

2.2 Realizability as Analysis over Regular Codata

Next, we show how to formulate and solve the realizability problem to which completability of the prefix 1+2 reduces. We split this problem into three goals:

(1) **Representation:** How can we finitely describe infinite program spaces?
(2) **Completion:** Given a string ω, how can we construct the space of all ASTs corresponding to completions of ω?
(3) **Analysis:** Does there exist a program in the constructed space satisfying the constraint?

Our next key insight is that **regular codata provides a structured abstraction for reasoning about spaces of programs**. Our solutions to (1)-(3) are built around this idea. In the rest of the section, we describe our approach following the overview given in Figure 2.

Goal 1: Representing Infinite Program Spaces. In our example, programs consist solely of integer literals and sums. To represent an AST for a single program, we can use the following datatype:

```
data Expr = Lit String      -- a numeric literal, e.g., Lit "5"
          | Sum Expr Expr   -- a sum of two expressions
```

We would like to lift this definition to represent (possibly infinite) *spaces* of ASTs. The first step is to extend the signature of `Expr` as follows:

```
data ExprSpace = Empty
               | Union [ExprSpace]
               | Lit Regex
               | Sum ExprSpace ExprSpace
```

[1]While the constraints being enforced in this example are task-specific, in practice users would write pruners for general constraints that apply across all tasks in an application domain (as in Section 5).

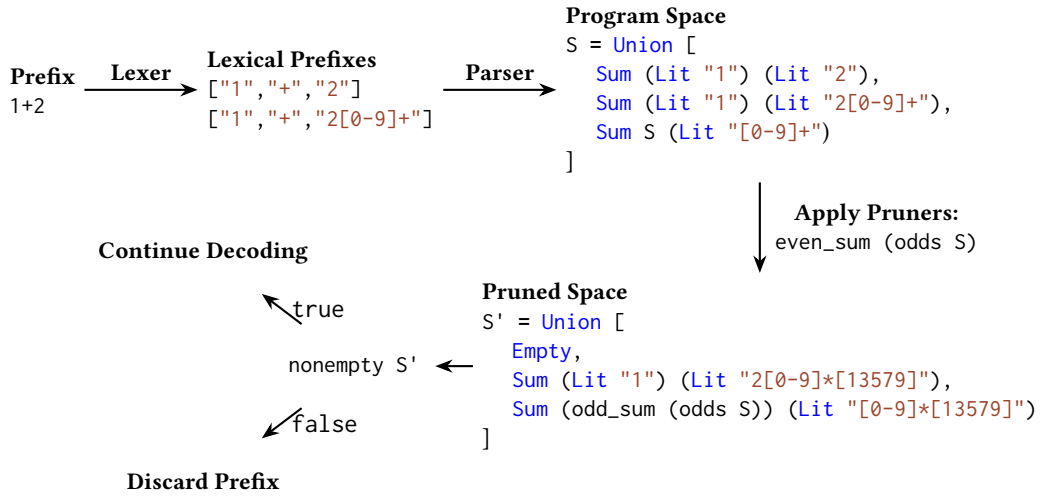


Fig. 2. Flow of ChopChop on input prefix 1+2. The prefix is lexed into possible lexical sequences, parsed into a symbolic program space, semantically pruned, and checked for nonemptiness to determine realizability. If realizble, the prefix may be extended. If unrealizable, the prefix is discarded.

An `ExprSpace` can be empty, a union of spaces, a regular expression that represents a *set* of concrete literals, or an application of the `Sum` constructor lifted to the product of two program spaces. Readers familiar with *version space algebras* [26, 39] will recognize this representation as a version space, where `Union` is the union node and `Sum` is a join node.

For spaces to represent infinite sets, we must generalize the shape of terms and allow corecursion. We represent program spaces as *regular coterms*, i.e., infinite, cyclic terms with a finite description. For example, the space consisting of *all* sums of integers can be represented using the following corecursive definition:

```
all =
    let int = Lit "[0-9]+" in
    Union [int, Sum all int]
```

Note that even though `all` is self-recursive, it has a *finite* representation in memory as a cyclic term in which the definition of `all` contains a back-reference to itself. Coterms that admit a finite representation (but are potentially infinite) are called regular.

In ChopChop, all program spaces are regular and represented using this finite, cyclic form (Section 4). We implement and manipulate coterms via a solver inspired by CoCaml [20], an extension to OCaml that supports equational reasoning for regular coterms. This solver can perform computations over program spaces—e.g., applying transformations such as `odds` and `even_sum` (Figure 3)—without materializing infinite sets.

Goal 2: Computing Prefix Spaces. Given a string prefix ω, we need to compute the space of all ASTs that can be produced by parsing any completion of ω. This construction is done with reference to a user-supplied parser definition that defines a grammar for the language as well as how to convert parse trees to ASTs (Figure 1).

As a first step, we lex ω into a finite set of *lexical prefixes* that encode how ω can be lexed once its last lexeme has been completed. For example, the prefix ω = 1+2 has two valid lexical prefixes, representing the different ways ω could be lexed depending on the next character:

- `["1","+","2"]`: where the next character is non-numeric (as in 1+2+).
- `["1","+","2[0-9]+"]`: where the next character continues the numeric literal (as in 1+21).

Next, for each lexical prefix we construct a corresponding program space representing its completions. This construction can be done naturally following the *parsing with derivatives* methodology [28]. In our framework, parsers are regular coterms representing the set of possible parse trees producible from their current state[2]. Cycles in parsers directly mirror cycles in the underlying context-free grammar. Critically, parsers support a *derivative* operation that advances the state of a parser as if it read one additional lexeme.[3]

For each lexical prefix—e.g., `["1","+","2"]`—we start from the user-provided parser and apply successive derivatives for each lexeme in the prefix; this results in a parser that accepts exactly those programs that begin with the given lexical prefix. Finally, we convert each derived parser into a corresponding program space (essentially by discarding the information about concrete syntax) and combine the program spaces from different lexical prefixes using the `Union` constructor. For our example, the prefixes `["1","+","2"]` and `["1","+","2[0-9]+"]` together would induce the following program space:

```
S = Union [ Sum (Lit "1") (Lit "2"),
            Sum (Lit "1") (Lit "2[0-9]+"),
            Sum S (Lit "[0-9]+") ]
```

Goal 3: Analyzing Program Spaces. The final step towards evaluating realizability is to determine whether there exists a constraint-satisfying program in the prefix space computed in the previous step. Unfortunately, this problem is undecidable in general, even when the constraint is decidable for individual ASTs [23, 24]. To make the analysis tractable, we therefore turn to the canonical strategy in program synthesis: pruning. Specifically, users encode their constraints by defining *semantic pruners*, functions that take as input a space of programs and return (an approximation of) the subspace of constraint-satisfying ones. In ChopChop, just as program spaces are modeled as codata, pruners are modeled as corecursive functions.

Pruners can be composed to enforce conjunctions of constraints, allowing users to modularly define and reuse semantic constraints across tasks. For our running example, we supply two pruners (shown in Figure 3): `odds`, which removes programs with even literals, and `even_sum`, which removes programs whose total is odd. When we apply the two pruners in sequence, we obtain a new regular coterm `S'` that represents the subspace of even sums of odd literals:

```
S' = even_sum (odds S)
```

Since `S` is cyclic, evaluating `S' = even_sum (odds S)` eventually leads back to `even_sum (odds S)`. To avoid infinite recursion, our solver tracks previously expanded terms and creates a cycle in `S'` when we re-visit a subterm. For example, the inner term `odds S` expands until we reach the state:

```
odds S = Union [ Sum (Lit "1") Empty,
                 Sum (Lit "1") (Lit "2[0-9]*[13579]"),
                 Sum (odds S) (Lit "[0-9]*[13579]") ]
```

[2]We use the term *lexeme* for programming-language tokens to avoid confusion with LM tokens.
[3]This approach mirrors the definition of the Brzozowski derivative [7] of a language L with respect to characer u, which is $u^{-1}L = \{w \mid uw \in L\}$.

```
odds :: ExprSpace -> ExprSpace
odds Empty            = Empty
odds (Union children) = Union (map odds children)
odds (Lit regex)      =
    Lit (regex `intersect` "[0-9]*[13579]") -- only odd
odds (Sum left right) = Sum (odds left) (odds right)
```

(a) odds pruner: retains only programs using odd literals.

```
even_sum :: ExprSpace -> ExprSpace
even_sum Empty            = Empty
even_sum (Union children) = Union (map even_sum children)
even_sum (Lit regex)      =
    Lit (intersect regex "[0-9]*[02468]") -- only even
even_sum (Sum left right) =
    Union [ Sum (even_sum left) (even_sum right),
            Sum (odd_sum left) (odd_sum right) ]

odd_sum :: ExprSpace -> ExprSpace
-- Analogous to even_sum
```

(b) even_sum pruner: retains programs whose total evaluates to an even number.

Fig. 3. Example semantic pruners.

Finally, we must check if there exists a program in the pruned space. Note that checking emptiness is not the same as checking if the space is exactly the term `Empty`, since the presence of cyclic subterms means evaluation may not reduce the space to a normal form. ChopChop implements a function `nonempty :: ExprSpace -> Bool` that performs a fixpoint computation over the codata representation of its input. Again, because spaces are *regular* coterms, the fixpoint computation operates over a finite structure despite the fact that a space can represent infinitely many programs.

In our example, `nonempty` determines that the first two sub-spaces of the union in `S'` are empty while the third one is non-empty, and hence the union is non-empty.

Putting it All Together. Together, the four components—lexing, parser derivatives, user-defined pruners, and the nonemptiness check—can compute realizability for our running example as:

```
nonempty (even_sum (odds S))
```

This pipeline is modular and requires minimal effort from the user: they define a grammar and supply pruners for each constraint. ChopChop takes care of the rest—automatically enforcing semantic constraints as the LM generates code.

3 Semantic Constrained Decoding as Realizability Checking

In this section, we formalize the problem of semantic constrained decoding and explain how it is equivalent to the problem of checking whether a synthesis problem is realizable—i.e., whether a regular tree grammar accepts any programs that satisfy a given input specification. After Section 3.1 establishes some basic definitions, Section 3.2 gives a general formulation of constrained decoding

where the constraint is a predicate on strings. Section 3.3 defines the special case of semantic constrained decoding, where the predicate is induced by a constraint on ASTs. We also provide soundness and completeness theorems.

3.1 Preliminaries

Strings and Abstract Syntax Trees. Let Σ be a finite alphabet of characters. A *string* is a sequence $\omega \in \Sigma^*$, and the *completions* of ω are the strings that begin with ω. Let AST be a set of *abstract syntax trees* (of programs). Translation from strings to ASTs is defined by a function $\mathsf{parse}\colon \Sigma^* \to \mathsf{AST} \cup \{\bot\}$ that produces an AST from a string of characters (or $\bot$ if parsing fails).

Language Models and Decoding. Language Models (LMs) operate over short character sequences called *tokens*, rather than over individual characters. We define a *token vocabulary* $\mathbf{T}$, where a token $\tau \in \mathbf{T}$ is either a finite string of characters from Σ or the special token END. A token vocabulary $\mathbf{T}$ must be large enough to represent any string in Σ^* as a sequence of tokens.

We formalize a *language model* $\mathsf{lm}\colon \mathbf{T}^* \to \mathbf{T} \to [0,1]$ as a function that takes in a *prefix* $\tau_1, \ldots, \tau_{i-1} \in \mathbf{T}^*$ and returns a probability distribution over the next token $\tau_i \in \mathbf{T}$. Given a language model, we can iteratively generate a sequence of tokens, selecting each next token τ_i based on the model's distribution, conditioned on the previously generated tokens $\tau_1, \ldots, \tau_{i-1}$. This process is called *decoding*. Decoding can be done greedily (by choosing the most probable token at every step) or stochastically (e.g., via nucleus sampling [17]). Decoding stops when an END-terminated token sequence is produced.

3.2 Constrained Decoding

We begin with a general definition of *constrained decoding* (CD) that is not specific to semantic constraints. A *string constraint* Φ is a predicate on Σ^* that accepts some set of strings. For example, Φ may accept the members of some context-free language. We say that a string $\omega \in \Sigma^*$ is *completable* with respect to Φ if there exists a completion of ω satisfying Φ:

$$\mathsf{completable}(\omega, \Phi) = \exists \omega' \in \Sigma^*.\ \Phi(\omega\omega')$$

A *monitor* for a string constraint Φ is a function $\mathsf{monitor}_\Phi : \Sigma^* \to \mathsf{bool}$ that checks whether an input string is completable with respect to Φ. For a given string constraint Φ, *constrained decoding* maintains the invariant that the currently generated sequence of tokens is completable with respect to a monitor for Φ. Given a language model lm, a string constraint Φ, and a monitor for Φ, Algorithm 1 outlines how constrained decoding operates in its general form.

To abstract over the choice of the decoding strategy, Algorithm 1 uses an opaque worklist data structure that stores all generated token sequences whose proper prefixes are completable, along with the sequences' generation probabilities. In each iteration, the algorithm dequeues a sequence ω from the worklist. (Note that for sampling-based techniques dequeuing may be a probabilistic operation.) If ω is END-terminated, we return $\omega[:-1]$ if $\omega[:-1]$ satisfies Φ and discard ω otherwise (lines 6-7). If ω is not END-terminated, we check whether any completion of ω can satisfy Φ (line 8). If yes, we enqueue the possible continuations $\omega\tau$ for later review (lines 9-10). If no, we discard ω.

3.2.1 Soundness and Relative Completeness of Constrained Decoding. The guarantees we can make about Algorithm 1 depend on the quality of $\mathsf{monitor}_\Phi$. A monitor is (*i*) *sound* if every prefix the monitor accepts is completable: $\mathsf{monitor}_\Phi(\omega) \Rightarrow \exists \omega' \in \Sigma^*.\Phi(\omega\omega')$; and (*ii*) *complete* if all completable prefixes are accepted by the monitor: $\mathsf{monitor}_\Phi(\omega) \Leftarrow \exists \omega' \in \Sigma^*.\Phi(\omega\omega')$. Constrained decoding is *sound* if every string produced by Algorithm 1 satisfies the constraint Φ. Constrained decoding is *complete* if every string satisfying Φ can be returned by Algorithm 1 with non-zero probability.

Algorithm 1 Constrained Decoding

```
1:  Input: LM lm, string constraint Φ, monitor monitor_Φ
2:  Output: String ω such that Φ(ω) holds
3:  worklist ← [(ϵ, 1)]
4:  while worklist ≠ [] do
5:    ω, p ← worklist.dequeue()
6:    if ω[−1] = END ∧ Φ(ω[: −1]) then
7:      return ω[: −1]
8:    elif ω[−1] ≠ END ∧ monitor_Φ(ω) then
9:      for τ ∈ T do
10:       worklist.enqueue(ωτ, p ∗ lm(ω, τ))
11: return ⊥
```

Constrained decoding maintains a worklist of pairs of form (ω, p), where ω is a token sequence and p is its probability of being a prefix of a string drawn from the distribution induced by lm. At each iteration, a string ω is dequeued (Line 5). If the string is END-terminated, we remove END and return the string if it satisfies Φ (Lines 6-7). If the string is not END-terminated and is completable with respect to Φ, we enqueue its one-token continuations for later review (Lines 8-10). Unconstrained decoding corresponds to removing the green parts of the algorithm.

Lemma 3.1 (Soundness). *Constrained decoding is sound if $\Phi(\omega)$ can be checked soundly.*

This is easy to see, as the algorithm only returns outputs ω that pass the check $\Phi(\omega)$. Note that this condition ensures that Algorithm 1 is sound *even when the monitor is unsound.*

Unlike soundness, completeness often does not hold for common instantiations of worklist (e.g., greedy decoding, where the highest probability continuation of the previously dequeued string is always selected). However, we still want a relative characterization of completeness to ensure that constrained decoding cannot discard solutions we could achieve with unconstrained decoding. To this end, we define a decoding strategy as simply an implementation of worklist. We say constrained decoding is *complete up to a decoding strategy* if under that implementation, Algorithm 1 can return every valid solution that unconstrained decoding can produce with at least as high probability.

We prove completeness up to two of the most popular decoding strategies, top-p and top-k sampling (though our proof methodology generalizes to other common strategies). Top-k sampling is a decoding method where, at each step, the model considers only the k most likely next tokens for its current prefix and randomly selects one based on their normalized probabilities. Top-p sampling is similar, except instead of a fixed k we use the smallest set of most probable tokens whose cumulative probability mass is above threshold p. Either of these sampling approaches could be implemented as instantiations of Algorithm 1 by setting dequeue to iterate over the worklist, only choose pairs where the token sequence has a maximal length, and sample from the appropriate subset after renormalizing probabilities.

Lemma 3.2 (Completeness up to Top-p and Top-k Sampling). *If monitor_Φ is complete, then constrained decoding (Algorithm 1) is complete up to both top-p and top-k sampling.*

Note that not all decoding strategies satisfy completeness up to decoding. In a greedy/deterministic beam search for example, discarding an incompletable prefix might cause the LM to explore a completable prefix it would ordinarily have neglected. This exploration may cause constrained decoding to find a different solution than unconstrained decoding would have originally found.

While complete monitors guarantee completeness up to common decoding strategies, sound monitors make search more efficient. In existing approaches for *syntactic constrained decoding*, the string constraint Φ is limited to asserting membership in a context-free language. In this case, monitors can be sound and complete [36, 41].

3.3 Semantic Constrained Decoding

Semantic constrained decoding is the special case of constrained decoding where the string constraint is defined as $\Phi = \phi \circ \mathsf{parse}$ for some predicate $\phi\colon \mathsf{AST} \to \mathsf{bool}$. We call ϕ a *semantic constraint*, We implicitly extend the domain of any such ϕ to $\mathsf{AST} \cup \{\bot\} \to \mathsf{bool}$ by setting $\phi(\bot) = \mathsf{false}$. For example, the semantic constraint in Section 2 checks whether an AST is an odd sum of even literals. The corresponding string constraint checks whether a string parses to such an AST.

Algorithm 1 requires a monitor for checking completability of the string constraint Φ. For semantic constrained decoding, we have that

$$\mathsf{completable}(\omega, \Phi) \equiv \exists \omega'.\ \phi(\mathsf{parse}(\omega\omega'))$$

The key insight of our approach is that we can re-express $\mathsf{completable}(\omega, \Phi)$ in terms of the *space* of ASTs a prefix ω represents. First, we define the *prefix space* of a string ω as the set of all parses of continuations of ω:

$$\mathsf{PrefixSpace}(\omega) \triangleq \{\mathsf{parse}(\omega\omega') \mid \omega' \in \Sigma^*\}$$

By changing the quantified variable to range over ASTs, completability can now be written as

$$\mathsf{completable}(\omega, \Phi) \equiv \exists t \in \mathsf{PrefixSpace}(\omega).\ \phi(t)$$

We are now asking if there exists a term in a given prefix space that satisfies a constraint ϕ. Readers familiar with program synthesis will recognize this formulation as the standard shape of a synthesis problem. Specifically, such a query is an example of a *realizability* [19, 23] problem, which asks whether a set contains a term that satisfies a given property.

$$\mathsf{realizable}(T, \phi) \triangleq \exists t \in T.\ \phi(t)$$

Expressing realizability entirely over spaces of ASTs removes the need for the string-level reasoning that has characterized prior constrained-decoding work [1, 31]. All together, our definition of completability simplifies to:

$$\mathsf{completable}(\omega, \Phi) \equiv \mathsf{realizable}(\mathsf{PrefixSpace}(\omega), \phi)$$

Consequently, a monitor for Φ can be implemented as $\mathsf{check_realizable}_\phi \circ \mathsf{PrefixSpace}$, where $\mathsf{check_realizable}_\phi$ is a potentially approximate procedure for checking realizability.

3.3.1 Soundness and Relative Completeness of Semantic Constrained Decoding. A realizability checker is (*i*) *sound* if every space it accepts is realizable: $\mathsf{check_realizable}_\phi(T) \Rightarrow \exists t \in T.\ \phi(t)$ and (*ii*) *complete* if every realizable space is accepted by the checker: $\mathsf{check_realizable}_\phi(T) \Leftarrow \exists t \in T.\ \phi(t)$.

Since PrefixSpace can be computed exactly, the following theorems follow immediately from Lemma 3.1 and Lemma 3.2:

Theorem 3.3 (Soundness of Semantic Constrained Decoding). *Semantic constrained decoding is sound if ϕ can be checked soundly.*

Theorem 3.4 (Completeness up to Top-p and Top-k Sampling of Semantic Constrained Decoding). *If $\mathsf{check_realizable}_\phi$ is complete, then semantic constrained decoding is complete up to both top-p and top-k sampling.*

Section 2 showed an example of a sound and complete realizability checker for the constraint being enforced. In Section 5, we present two more realistic examples: a sound and complete realizability checker for the problem of equivalence-guided decoding, and a complete[4] realizability checker for type-safe decoding.

4 ChopChop: Semantic Constrained Decoding via Coinduction

The key idea of semantic constrained decoding is that one can define pruners on *spaces* of programs, which are defined at the level of abstract syntax. To create a programmatic interface for defining pruners, we need a framework for constructing, manipulating, and performing analyses over possibly infinite program spaces. One framework for cleanly manipulating infinite objects is supplied by Jeannin et al. [20]; the framework defines infinite objects with repeated structure (like program spaces) as *regular codata* and lets users compute recursive functions on regular codata (Section 4.1). We adopt this framework to represent and manipulate program spaces (Section 4.2). As we show in Section 4.3, framing program spaces as regular codata allows us to not only implement pruners over program spaces, but also gives an elegant way to construct prefix spaces. Finally, we describe how ChopChop implements this approach as an embedded DSL in Python (Section 4.4).

4.1 Programming with Regular Coinduction

Coinduction is a powerful technique for representing and manipulating infinite objects. We present a brief introduction to coinduction adapted from the running example used by Jeannin et al. [20].

Codata. Consider the usual datatype for integer lists:

```
data IList = INil
           | ICons Int IList
```

The `IList` datatype includes finite lists like `INil` and `ICons 1 (ICons 2 INil)`. However, `IList` also includes *infinite* streams like, e.g., `ICons 1 (ICons 2 (ICons 3 ...))`. Infinite members of a datatype are called *coterms*, or sometimes *codata*.

In general, it is not always possible to finitely represent a coterm. However, many coterms do admit compact representations. For example, the stream of all ones can be written as `ones = ICons 1 ones`. The stream `ones` is an infinite term, but it only has finitely many distinct subterms (i.e., `1` and `ones`). This property, called *regularity*, allows codata to be expressed with a finite representation. Alternatively, regular coterms can be depicted as cyclic term graphs, as shown by Figure 4.

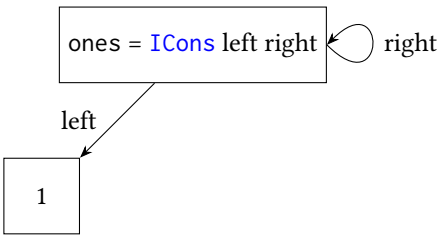


Fig. 4. Structure of the ones stream.

Because regular codata can be finitely represented, we can compute their properties in finite time. For example, one cannot compute the set of integers contained in an arbitrary stream. However, a quick inspection of the definition of `ones` (or a fixpoint procedure over the term graph) reveals that `ones` contains no integers besides 1. When we represent program spaces as regular codata later, we will use these nice properties to answer questions about, e.g., the nonemptiness of a program space.

[4]For this problem one can prove that no sound and complete realizability checker exists [44]

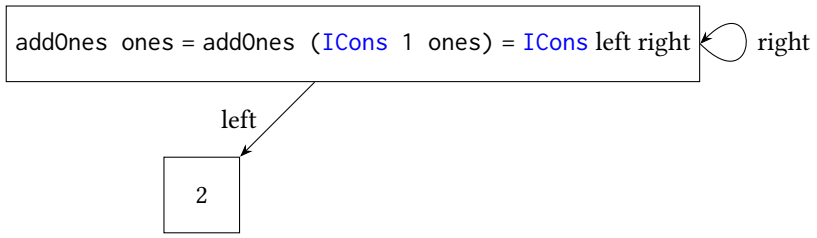


Fig. 5. Structure of addOnes ones.

Computations over Regular Codata. To describe operations on codata, we can define typical recursive functions on the structure of the datatype representing codata. For example, we can define the following function to collect the set of distinct integers appearing in a stream:

```
@fixpoint: EmptySet
elements :: IList -> Set Int
elements INil = EmptySet
elements ICons x xs = (add (elements xs) x)
```

Over an arbitrary infinite coterm `t`, it may not be possible to explicitly compute `(elements t)`. However, one can do so when `t` is regular. The key observation is that, if `t` has only finitely many distinct subterms, we will only need to evaluate `elements` on finitely many distinct coterms. In general, functions that compute data from regular coterms can be evaluated as fixpoint computations over the coterm's cyclic term graph. Throughout the paper, we distinguish such functions with the annotation **`@fixpoint`**, followed by the initial value for the fixpoint computation.

It is also possible to recursively define functions that produce regular coterms. In the case of streams, we might consider a function that adds 1 to each element of the stream

```
addOnes :: IList -> IList
addOnes INil = INil
addOnes (ICons x l) = ICons (x + 1) (addOnes l)
```

If we apply `addOnes` to `ones`, we see that only finitely many distinct terms are generated:

```
addOnes ones = addOnes (ICons 1 ones)
             = ICons 2 (addOnes ones)
```

Regular corecursive functions (i.e., functions that map regular coterms to regular coterms) can be computed explicitly over regular coterms with the techniques of Jeannin et al. [20]. At a high level, their techniques work by memoizing the result of function expansions and producing a cycle upon encountering a previously seen call.

Of course, it is possible that the user may write a function that yields a coterm that is *not* regular. We do nothing to prevent this or otherwise enforce termination—nontermination is a wholly orthogonal problem to our purposes that has been studied in the literature on coinduction, for example in [8]. In practice, one can avoid most issues by (*i*) only writing recursive functions that call themselves on subterms of the input term and (*ii*) avoiding **`@fixpoint`**-annotated functions whose return types contain infinitely many values. Finally, our framework provides one special built-in function, `isCyclic`, that returns true if a regular coterm has cycles and false if it is just a tree.

4.2 Program Spaces as Coterms

In Section 2 we lifted the `Expr` datatype, which describes individual programs, to `ExprSpace`, which describes spaces of programs. Now, we generalize that principle to arbitrary program datatypes. First, we define a generic `Program` datatype (Figure 6a) that describes the structure of abstract syntax trees. It is parametric over a type parameter `sym`, which represents the different possible constructors for ASTs. We write `Program` this way to allow for manipulations over ASTs that are generic over the constructor. Often, we want to treat a datatype like `Expr` "as if it were" an instance of `Program`. That is, `Expr` is roughly the same as `Program ExprSym`, where `ExprSym = Sum`, ignoring constraints on arity of children. For notational convenience, we ignore this distinction and treat these two datatypes as the same.

We can abstract over `ExprSpace` in the same way we abstracted over `Expr`. Specifically, we can define a `ProgramSpace` (Figure 6b) as either: `Empty`, the empty space representing no programs; `Union`, a union of program spaces; `Lit`, a set of concrete literals described by a regular expression; or `ASTNode`, a product of program spaces under some AST constructor. We will write most concrete program spaces in the style of `ExprSpace` for clarity, but generic algorithms on program spaces will be written over `ProgramSpace`.

As discussed in Section 2, we need to be able to decide whether a `ProgramSpace` represents the empty set. While `Empty` is obviously empty, determining emptiness of a coterm like `E = Union [E, E]` requires a fixpoint computation. To solve this problem, we implement a simple function, `nonempty`, to check whether a space is empty:

```
@fixpoint: False
nonempty :: ProgramSpace sym -> ProgramSpace sym
nonempty Empty = False
nonempty Union children = any nonempty children
nonempty (Lit l) = regexNonempty l
nonempty ASTNode _ children = all nonempty children
```

Finally, when writing pruners it will be useful to convert a `ProgramSpace` representing only a single program into the corresponding element of type `Program`. Such spaces naturally arise when prefixes contain complete subexpressions. For this reason, we define a utility function `collapse` as:

```
collapse :: ProgramSpace sym -> Maybe (Program sym)
collapse x | isCyclic x     = Nothing
collapse (Union [x])        = collapse x
collapse (Lit l)
  | isSingleton l           = Just (ASTLeaf (toString l))
collapse ASTNode sym [children] = do
    collapsedChildren <- mapM collapse children
    ASTNode sym collapsedChildren
collapse _                  = Nothing
```

4.3 Parsers as Coterms

Recall from Section 2 that our lexer takes a string ω and returns a finite set of *lexical prefixes*. A lexical prefix is a sequence of complete lexemes followed by a possibly incomplete lexeme. The lexer is designed so that every lex of a completion of ω extends one of the lexical prefixes. The details of our lexing algorithm are discussed in the extended version of the paper [33].

```
data Program sym =
  | ASTLeaf String
  | ASTNode sym [Program sym]
```

(a) `Program` datatype, parametric over a type `sym` of AST constructors.

```
data ProgramSpace sym =
  | Empty
  | Union [ProgramSpace sym]
  | Lit Regex
  | ASTNode sym [ProgramSpace sym]
```

(b) Lifting of the `Program` datatype to a `ProgramSpace`.

Fig. 6. Programs and program spaces.

Once lexing is complete, we must perform parsing. We carry out parsing by following the *parsing with derivatives* technique introduced by Might et al. [28]. Intuitively, a *parser* is a function from strings to trees that represents the state of a parser after consuming some string ω. Parsers support a *derivative* operation that advances the parser state by consuming a lexeme, analogous to Brzozowski derivatives for regular expressions [7]. Parsers, their derivative operation, and a method to convert a parser to a program space can be easily represented as codata and corecursive functions.

Intuitively, after consuming some prefix ω, a parser (Figure 7) can be in one of a few possible states when it considers how to handle further lexemes.

- `EmptyParser`: The parser accepts no strings, meaning ω is not in the language that the parser recognizes.
- `Choice ps`: The parser accepts any string accepted by any parser in the list of parsers ps.
- `ConstParser reg`: The parser accepts any single lexeme in the regex reg.
- `NullParser l`: The parser has parsed the lexeme `l` but will consume nothing further.
- `Seq a ps1 ps2`: The parser is a sequential composition of parsers, where `ps1` is the sequence elements whose parsing is complete and `ps2` is the sequence of elements that remain. The *action* `a` describes how completed parses map to AST nodes.

4.3.1 Derivatives on Parsers. To advance the state of a parser, we define a *corecursive derivative function* `d` (Figure 8a). Given a lexeme l and a parser p, the coterm `d l p` represents the parser that is obtained after the parser `p` consumes the lexeme l. The definition of `d` recurses on the structure of the input parser `p`:

- If p is an `EmptyParser`, then `d p l` returns `EmptyParser`.
- If p is a `NullParser`, then `d p l` returns `EmptyParser` as p cannot accept further input.
- If p is a `ConstParser`, then `d p l` returns a constant parser storing l if the regex for p matches l; otherwise it returns `EmptyParser`.
- If p is a `Choice`, then `d p l` returns a `Choice` of the derivative of its children.
- If p is a `Seq`, then `d p l` derives the parser at the current position in the sequence and splits into a `Choice` of cases. If the derived parser can accept the empty string, we advance to the next parser in sequence by applying the nullability operator `delta`, which "completes" a parser by throwing away all parses that haven't been fully matched. Otherwise, we update the sequence in place.

More details appear in Might et al. [28] and Darragh and Adams [12].

4.3.2 From Parsers to Program Spaces. Finally, we need to construct a program space reprsenting all programs our parser could emit. To do so, we define a corecursive function `image` (Figure 9) that recurses over the structure of the input parser `p`:

```
data Parser sym =
  | EmptyParser
  | Choice [Parser sym]
  | ConstParser (Regex reg)
  | NullParser (String l)
  | Seq (Action sym)
        [Parser sym] [Parser sym]
```

(a) `Parser` datatype, parametric over the the type parameter `sym` describing the AST constructors it can produce.

```
data Action sym = Action {
    constructor : Maybe sym,
    child_indices: [Int]
}
```

(b) An `Action` describes how a sequence of parse trees is converted into an abstract syntax tree: it stores the AST constructor to use along with a list of indices of children to pass in.

Fig. 7. Parsers and actions.

```
d :: Lexeme -> Parser sym -> Parser sym
d l (ConstParser reg)      =
  if matches reg l
    then NullParser l
    else EmptyParser
d l (Choice ps)            =
    Choice (map (d l) ps)
d l (Seq a finished p::ps) =
  let derived = d l p in
  Choice [
    Seq a (finished ++ [delta derived]) ps,
    Seq a finished (derived : ps)
  ]
d _ _                      = EmptyParser
```

(a) Derivative for parsers: in the `Seq` case, we use the helper δ to handle the case where the current parser in sequence has finished consuming input.

```
delta :: Parser sym -> Parser sym
delta p@(NullParser _)  = p
delta (Choice ps)       =
    Choice (map delta ps)
delta p@(Seq _ _ [])    = p
delta _                 = EmptyParser
```

(b) The function δ returns the "nullable" portion of a parser, i.e. the branches which do not consume further input.

Fig. 8. Definition of derivative for parsers.

- If p is an `EmptyParser`, then `image` returns the empty space.
- If p is a `Choice`, then `image` returns a `Union` of the images of its children.
- If p is a `ConstParser`, then `image` returns a `Lit` matching any lexeme p could accept.
- If p is a `NullParser`, then `image` returns a `Lit` matching only the lexeme p has already accepted.
- If p is a `Seq`, then `image` applies the `Action` to p. If the constructor for the action is `None`, we "skip" converting this layer of the parser—this option supports inlining trees for constructs like parenthesis specifically.

Together, the lexing, derivative-based parsing, and image construction enable ChopChop to track the evolving set of valid program completions at every step of generation. This coinductive representation forms the foundation for subsequent semantic pruning and realizability checking.

```
image :: Parser sym -> ProgramSpace sym
image EmptyParser = Empty
image (Choice ps) = Union (map image ps)
image (ConstParser reg) = Lit reg
image (NullParser l) = Lit l
image (Seq a ps ps') =
    let children = ps @ ps' in
    match a.constructor with
        Nothing ->
            image $ children!!(head a.child_indices)
        Some s  ->
            ASTNode s [image $ children!!i | i <- a.child_indices]
```

Fig. 9. The image of a parser is the space of trees it could emit.

4.4 Implementation

We implemented ChopChop as an embedded domain-specific language (DSL) in Python that supports symbolic manipulation of coinductive structures. The backend draws inspiration from prior work on regular coinduction, such as CoCaml [20], and makes extensive use of Python metaprogramming. We do not use CoCaml directly to ensure compatibility with the rest of our Python implementation. Notably, ChopChop defines two key decorators:

- `@rewrite`: Marks functions that construct coterms by symbolically rewriting recursive calls into a finite system of equations (if the recursion is regular).
- `@fixpoint`: Interprets a coterm via Kleene iteration to compute a fixed point over the coterm's input.

Additionally, ChopChop incorporates domain-specific simplifications for program spaces. For instance, unions over parser states or AST spaces are simplified by removing empty branches—i.e., when `nonempty` returns false. For parser derivatives, we also implement the compaction optimization described in [28] to reduce the size of coterms.

5 Case Studies

We demonstrate the generality of our framework by instantiating it for two domains: enforcing semantic equivalence for a basic functional language and enforcing type safety for TypeScript.

5.1 Equivalence-Guided Decoding

In this case study, the LM is given expressions in a basic functional language and is asked to refactor them into equivalent programs. The language consists of basic arithmetic operators, identifiers, integer constants, function applications, and let bindings. For example, consider the following program that computes the distance between two points.

```
sqrt (pow (x2 - x1) 2 + pow (y2 - y1) 2)
```

A valid output for the LM might be:

```
let dx = x2 - x1 in
let dy = y2 - y1 in
sqrt (pow dy 2 + pow dx 2)
```

In general, program equivalence is not decidable; therefore, ChopChop uses *e-graphs* [46] to reason about program equivalence approximately. An e-graph is a data structure for compactly representing spaces of equivalent programs. At a high level, an e-graph maintains a collection of equivalence classes of expressions. To build an e-graph, a user supplies an initial expression and a list of *rewrite rules* that define equivalences. For example, the rule $?x+?y \rightarrow ?y+?x$ encodes commutativity of addition. A term like $f(a, b) + 2$ can *match* the left-hand side of this rule, to be rewritten to $2 + f(a, b)$. Initially, an e-graph only represents trivial equivalences (i.e., every equivalence class has only one member). Equivalence classes are then refined via a process known as *equality saturation*, which iteratively applies rules (up to a fixed budget) to subterms in the e-graph. Whenever a subterm matches a rule, the e-graph is modified by applying the rewrite and merging the equivalence classes of the rewritten subterm with the subclass of the original subterm. Crucially, e-graphs can compactly represent large sets of terms by *sharing*: instead of storing a term like $f(x, y)$ directly, an e-graph will add a layer of indirection so that the node f has pointers to the *equivalence classes* of x and y, rather than the expressions themselves.

For our case study, we define a set of basic rewrite rules for arithmetic expressions (ones with addition, subtraction, multiplication, and division; see the extended version of the paper [33] for the full set of rules). We then use the egglog library [47] to build an e-graph. ChopChop then enforces the constraint that the output of an LM must be a program stored by this e-graph—i.e., ChopChop guarantees that the output term is equivalent up to rewrite.

Importantly, the set of terms an e-graph represents forms a regular tree language that can be represented by a finite tree automaton [40]. We use this property to build a semantic pruner whose goal is to only keep terms that appear in the automaton. In particular, we syntactically convert the automaton corresponding to the e-graph into a corresponding `ProgramSpace` coterm. Then we can define a function `intersectSpaces` to filter out non-equivalent programs from the prefix space.

While e-graphs can easily deal with let-free expressions, we need some extra care to make sure that let-expressions are taken into account by the e-graph. To handle let expressions, our pruner adopts a simple strategy: if we have not finished generating a let binding, we do not prune any programs—no matter what expression the let binds a variable to, we can always avoid using the variable in the final expression. If a let expression space contains a complete binding of form $x = e_1$ where e_1 is a complete expression (i.e., the LLM has generated a let binding), then we update the current e-graph with an additional rewrite rule $e_1 \rightarrow x$. When generating the last expression under the let expression, we use the semantic pruner induced by this new e-graph. The pseudocode for our checker is shown below:

```
prune (Let b s1 s2) egraph = fromMaybe Empty do
    x <- collapse b       -- Wait for variable name to be generated
    e1 <- collapse s1     -- Wait for binding for x to be generated
    let egraph' = updateEGraph egraph x e1
    prune s2 egraph'      -- Recurse, adding extra rule e1 => x
prune s egraph             = s `intersectSpaces` (eGraphToSpace egraph)
```

5.2 Type-Safe Decoding for TypeScript

In our second case study, we use ChopChop to implement a semantic pruner that enforces type safety for a subset of TypeScript that covers most core language features (branching, loops, variable declaration and reassignment, constant and mutable variables, and [recursive] function definitions). We study TypeScript because it is widespread [15], and previous work has shown that small and mid-sized LMs often fail to produce type-safe TypeScript code [31].

It is a folklore result that even for simply-typed lambda calculus, the set of well-typed terms is not a regular tree language. As a consequence, no pruner can remove exactly the ill-typed programs from a program space—the pruned space would not be representable as regular codata. Instead, we implement an overapproximate pruner that conservatively discards many ill-typed programs while ensuring that all well-typed programs are preserved.

Type Pruning. Our typesafety pruner is a corecursive function of the following form:

```
type_prune :: Environment -> Type -> ProgramSpace -> ProgramSpace
```

Much like a type checker, `type_prune` is parameterized by a typing environment and target type. However, it operates on program spaces instead of single programs: given an input space it returns an overapproximation of the subspace of well-typed programs. For example, for the program space `(Union (Add x 6) false)`, the expression

```
type_prune {x: int} int (Union (Add x 6) false) = Add x 6
```

yields a program space containing `(Add x 6)`, the only program that types to `int` when the variable `x` has type `int`. If we call `type_prune` with the empty environment and the target type $\top$ (i.e., the supertype of all types), we get an approximation of the set of well-typed closed programs—for the program space `(Union (Add x 6) false)`, we would get `false`. The extended version [33] of the paper discusses the full implementation of `type_prune`; we give an overview below.

Designing `type_prune`. We designed `type_prune` to be an analogue of checking in bidirectional type checking [37], which resolves typing judgements locally and is deterministic. For ground terms, we check their type against the target type and decide whether to keep them or prune them (yielding the `Empty` program space):

```
type_prune {x: int} int  6 = 6
type_prune {x: int} bool 6 = Empty
```

For most program constructors, a bidirectional type checker would check the types of the subterms to decide how to type the superterm. Similarly, our type pruner prunes each set of subterms and returns only the valid combinations:

```
type_prune {x: int} int (Add x 6) =
    Add (type_prune {x: int} int x) (type_prune {x: int} int 6)
```

Like with bidirectional type checking, this approach gets complicated for function applications. Suppose `F` and `X` are program spaces, and consider type pruning the application of `F` to `X`—i.e., `(type_prune {} int (App F X))`. We would like to handle this case by pruning `F` and `X` to their expected types and combining the results. However, there are infinitely many valid combinations of types that programs in `F` and `X` could take, e.g., `int -> int` paired with `int`, or `(bool -> int) -> int` paired with `bool -> int`, and so on.

In bidirectional type checking, this multiplicity is resolved by first inferring the type of the function. When the type of the function is known, the argument can be checked against the function's input type. Since `F` may represent infinitely many programs with infinitely many possible types, type inference on `F` may be insufficient to resolve the multiplicity. Instead, we *wait* until `F` resolves to a single AST (i.e., via `collapse` from Section 4.2). Before `F` resolves to a single AST, we type prune `F` to exclude functions that cannot return the target type, and we leave `X` unmodified. Once `F` resolves to a single AST, we can infer the type of that AST and use it as the basis of our pruning. For example, if `F = foo`, then

```
type_prune {foo: (int -> int) -> int} int (App foo X) =
    App foo (type_prune {foo: (int -> int) -> int} int -> int X)
```

Type inference for single ASTs can be implemented in the usual way (e.g., [5]). type pruning `Empty` always yields `Empty`, and type pruning a `Union` will type prune each child and union the results.

Despite the overapproximate handling of function application, our pruner is quite precise over program spaces that correspond to left-to-right program prefixes. For such spaces, `F` will always resolve to a concrete function before any of `X` is written. This left-to-right resolution means that we can always infer the type of `F` when we start writing `X`.

We note that ChopChop makes it more natural to implement an overapproximate type pruner than an underapproximate type pruner. The idea of waiting for a term to be completed is supported naturally by the `collapse` function. In contrast, writing an underapproximate pruner requires analyzing some subset of the combinations of function and argument types at the same time.

6 Evaluation

We evaluate the two instantiations of ChopChop presented in Section 5 to answer the following research questions:

RQ1) Effectiveness. Does ChopChop make language models more effective at generating programs that satisfy semantic constraints?

RQ2) Overhead. What is the computational cost (e.g., latency, token usage) of enforcing semantic constraints during generation?

6.1 Experiment Setup

We compare our semantic constrained decoders written in ChopChop to the following baselines:

- ***Unconstrained Decoding***: The LM generates code without any constraints.
- ***Grammar-Constrained Decoding (GCD)***: The LM must produce syntactically valid programs (enforced via a grammar), but no semantic restrictions are applied. In ChopChop, grammar-constrained decoding corresponds to using the identity function as the pruner.

Benchmarks: Equivalence-Guided Decoding. We created 10 benchmark tasks in the basic functional language shown in Section 5.1, where the goal is to refactor a program into an equivalent one—e.g., factoring out subexpressions into **`let`** bindings. We use a fixed system prompt that specifies the grammar of the language and instructs the model to return only a refactored program[5] with no explanation. The exact prompt and the full list of benchmarks are shown in the extended version of the paper [33]. We count a response as *correct* if the LM produces a complete program that is equivalent to the input program, with no post-processing. Unconstrained decoding often fails to produce just the output code, despite being explicitly instructed to do so. Therefore, we also evaluate a variant where the prompt wraps outputs in triple backticks (```) to encourage the model to delimit its code clearly. This variant avoids penalizing runs that fail only due to formatting. We refer to the two variants as *No Delimit* and *Delimit*, respectively.

Benchmarks: Type-Safe Decoding. For our second instantiation of ChopChop, we source the benchmarks from the TypeScript translations of the MBPP [3] tasks from the MultiPL-E dataset [9]. We extract the 72/809 tasks that can be solved in our language fragment. Our prompting strategy mirrors that of Mündler et al. [31], but we additionally provide context to the LM to instruct it to avoid language constructs outside our language fragment. Our full prompting strategy and benchmarks are included in the extended version of the paper [33].

[5]We also ran a variant of this benchmark where we included in the prompt the list of rewrite rules. The overall trends are similar; this data can be found in the extended version of the paper [33].

An example task is given below:

```
// Write a typescript function to find the next perfect square
// greater than a given number.
function next_Perfect_Square(N: number): number
```

We count a response as *correct* if the generated TypeScript program compiles. For the TypeScript evaluation, we also run the tool by Mündler et al. [31] for comparison (although we do not instruct the LM to avoid programs outside our language fragment since their tool supports a larger subset of TypeScript).

Models, Parameters and Hardware. We run all experiments using the instruction-tuned versions (i.e. models that are trained to follow instructions in the prompt) of the following models: DeepSeek-Coder-6.7b, CodeLlama-7B, and CodeLlama-13B. Each model is evaluated at five different sampling temperatures: 0.01, 0.3, 0.5, 0.7, 1.0. These ranges of small-to-medium models and low-to-high temperatures lets us explore a range of model capability. We run all experiments on a Supermicro SYS-4029GP-TRT with two Intel(R) Xeon(R) Gold 6230 CPUs, 384 GB RAM, a 4 TB SSD, and eight Nvidia Geforce RTX 2080Ti GPUs.

Procedure. For each benchmark, model, decoder, and temperature, we run constrained decoding until either: the END token is generated or a 150 second timeout is reached. We only implement a naive sampling strategy where if a token is rejected, we backtrack by one token and re-sample with that token removed. Constrained decoders may fail if they use up their token or time budget without completing a valid program—especially if the LM repeatedly proposes unrealizable tokens that must be pruned. Unconstrained decoders may also fail to terminate if the model does not emit an END token withing the budget.

A run is considered **successful** if: (*i*) A complete program is emitted, and (*ii*) It satisfies the semantic constraints of the task (i.e., equivalence or type safety).

6.2 RQ1: Effectiveness

6.2.1 Quantitative Evaluation. Table 1 reports the number of successful runs for ChopChop and the two baselines—unconstrained decoding and grammar-constrained decoding—on all benchmarks. Across nearly all configurations, semantic constrained decoding delivers consistent and often dramatic improvements.

Equivalence-Guided Decoding. Unconstrained, most models perform very poorly on our benchmarks, with most failing to generate even a single semantically equivalent program. Several factors contribute to this: (*i*) We intentionally use small- and medium-sized models, which highlight the gains possible even for less capable models. (*ii*) The toy language used in the benchmarks is likely out-of-distribution for most pretrained LMs, which are tuned on real-world languages like Python and JavaScript. In particular, DeepSeek-Coder-6.7B frequently attempts to write Python, Lisp, or TypeScript code, and fails all benchmarks without semantic constraints as a result. (*iii*) Despite clear instructions, models frequently emit natural language explanations, markdown, or commentary—none of which are semantically valid outputs under our equivalence checker.

Because we define equivalence with respect to a reference set of rewrite rules, it is possible that these rules do not encode every possible equivalence a user might have in mind—e.g., in our benchmarks we treated all named functions as uninterpreted, but particular functions could have additional semantics associated with them. To mitigate this source of imprecision, we manually check the unconstrained and syntax-constrained data for false negatives (programs that could be considered equivalent, but are not proved equivalent by our rewrite rules). We only found 8 such instances out of the 600 total programs for these baselines.

Table 1. Successful generations for different decoding strategies across models and temperatures (higher is better). For equivalence-guided decoding we report the number of benchmarks for which an equivalent program was produced. For TypeScript we report the number of benchmarks on which compilable code was produced. Best results per column are **bolded**.

		DeepSeek-Coder-6.7b						CodeLlama-7B						CodeLlama-13B					
		Temperature					Tot.	Temperature					Tot.	Temperature					Tot.
		0.01	0.3	0.5	0.7	1.0		0.01	0.3	0.5	0.7	1.0		0.01	0.3	0.5	0.7	1.0	
Equivalence No Delimit (10 programs)	Unconstrained	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	Grammar	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	Semantic	**7**	**7**	**8**	**8**	**3**	**33**	**8**	**9**	**9**	**9**	**8**	**43**	**10**	**8**	**8**	**10**	**6**	**42**
Equivalence Delimit (10 programs)	Unconstrained	0	0	0	0	0	0	3	4	2	3	1	13	4	2	2	3	2	13
	Grammar	0	0	0	0	1	1	5	6	7	4	1	23	6	5	2	2	0	15
	Semantic	**10**	**10**	**8**	**9**	**8**	**45**	**9**	**10**	**10**	**6**	**6**	**41**	**9**	**8**	**7**	**8**	**6**	**38**
TypeScript (72 programs)	Unconstrained	57	55	48	45	9	214	69	71	71	66	31	308	68	67	60	47	17	259
	Grammar	38	31	28	19	10	126	67	65	62	59	36	289	68	67	57	36	8	236
	Semantic	**65**	62	61	58	28	274	**72**	71	**72**	66	58	339	**72**	69	69	66	43	319
	Mündler et al. [31]	62	**69**	**71**	**68**	**67**	**337**	67	**72**	70	**71**	**71**	**351**	70	**72**	**70**	**68**	**67**	**347**

Type-Safe Decoding. All three models often produce compiling code, except at high temperatures, where their performance degrades. The two CodeLlama models perform similarly, and the DeepSeek-Coder-6.7b model performs somewhat worse, especially at the highest temperature. Surprisingly, CodeLlama-7B slightly outperforms CodeLlama-13B at most temperatures. When aggregated across all temperatures, semantic constrained decoding consistently outperforms the baselines, improving the performance of DeepSeek-Coder-6.7b, CodeLlama-7B, and CodeLlama-13B by an average of 28.0%, 10.1%, and 23.2% across all temperatures, respectively. We also observe that grammar-constrained decoding alone actually performs worse than unconstrained generation. This effect is likely due to the fact that we limit the grammar to a small fragment of TypeScript and prevent programs that would compile, but are not in our supported fragment, from being generated. Additionally, we observe that Mündler et al. [31] performs slightly better than our semantic constrained decoding at medium temperatures and significantly better at the highest temperature. The reason for this discrepancy is that their tool supports a larger language fragment than we do, and the model tends to ignore the prompt and use forbidden language features as temperature increases.

6.2.2 Qualitative Evaluation. Next, we discuss some instances that show the strengths of semantic constrained decoding relative to GCD and unconstrained generation.

Equivalence. One interesting phenomenon is that while GCD ensures syntactic validity, it does not prevent models from creatively "embedding" invalid outputs within the grammar. For example, the text in 10a can be parsed (misleadingly) as a series of function calls due to the permissiveness of the grammar. This quirk allows LMs to "escape" into natural language. By contrast, semantic constrained decoding disallows such outputs because programs with undefined variables cannot be equivalent to the original. Next, we show a case where semantic constrained decoding prevents an inequivalent (but otherwise error-free) program from being generated. CodeLlama-7B under GCD returns the code in Figure 10b when given the program `start + (end - start) * scale`. This program needs to be extended with the suffix `+ start` to be correct. Semantic constrained decoding prevents the model from terminating the program string prematurely and would force it to continue (equivalence-constrained) generation.

TypeScript. Enforcing type safety prevents variable-name and function-name hallucinations, eliminating many errors that appear in the GCD and unconstrained modes. We observe that

```
Here is an equivalent refactored
version of the program using
explicit variable
```

(a) Natural language that parses as a valid program, generated with grammar-constrained decoding by CodeLlama-7B (see Section 5.1).

```
let start_scaled = start * scale in
let end_scaled = end * scale in
(end_scaled - start_scaled)
```

(b) Inequivalent program generated with grammar-constrained decoding by CodeLlama-7B, for initial program `start + (end - start) * scale` (see Section 5.1).

```
function is_woodall(x: number): boolean {
  const divisors: number = x / 25;
  let countDivisors: number = 0;
  while (divisors > 1) {
      if (x % divisors === 0) {
          countDivisors += 1;
          break;
      } else {
          divisors = divisors - 1;
      }
  }
  ...
}
```

(c) Snippet of a TypeScript program that mutates the constant variable `divisors`, generated during testing in grammar-constrained decoding mode by CodeLlama-7B at T=0.5 (see Section 5.2).

Fig. 10. Sample incorrect outputs generated when not using semantic constrained decoding.

at high temperatures the unconstrained mode also struggles to correctly balance parentheses, and GCD often calls undefined functions when it wants to use forbidden language features (e.g., `newArray(...)` is used instead of `new Array` to try to construct an array). A more interesting (but rare) case type-constrained decoding prevents is mutation of variables declared `const`. Consider the code in Figure 10c, produced with GCD. The variable `divisors` is declared constant on line 2, but it is mutated on line 9. In this case, the LM's mistake on line 2 does not cause an observable error until later in the program. The type system implemented in our semantic pruner will prevent the assignment to `divisors`, but it will still allow line 2.

When irresolvable conflicts between the LM's intention and the formal constraints of the decoder like Figure 10c occur, the LM is typically unable to recover because it has already "committed" to a solution. In the best case, this mistake results in continuations consisting of legal but useless code, such as whitespace or comments. In the absolute worst case, the LM will sometimes generate deeply nested arithmetic expressions that explode the size of our program space and slow our solver to the point of timing out. In general, constrained decoding is known to skew the LM token distribution and how to mitigate this problem is an active area of research [35]. Such adjustments are orthogonal to our purposes, so we do not consider them.

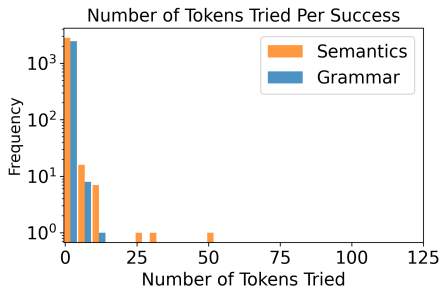
6.3 RQ2: Overhead

Table 2 shows the average overhead of semantic constrained decoding (in ms) per generated token. Overhead on decoding time range from tens to a couple hundred milliseconds per token, which is a very small price to pay for assurance provided by semantic constrained decoding. As a reference, on our hardware, CodeLlama-13B takes on average 81 ms to produce a token when unconstrained.

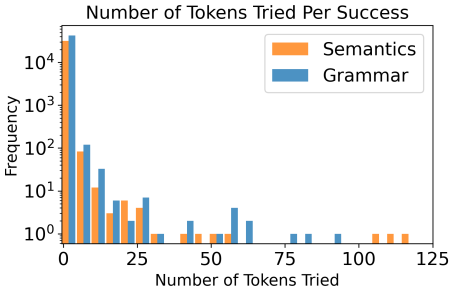
To better illustrate the source of the overhead, Figure 11 plots the distribution of the number of tokens that are tried before finding a realizable one, for CodeLLama-7B. Results for other models can be found in the extended version of the paper [33]. In general, even with semantic constrained decoding the first token tried is accepted most of the time. For instance, across all models, 96.4%

Table 2. Overhead of checking realizability in semantic constrained decoding (milliseconds/produced token).

Overhead in ms/token	DeepSeek-Coder-6.7b Temperature					CodeLlama-7B Temperature					CodeLlama-13B Temperature				
	0.01	0.3	0.5	0.7	1.0	0.01	0.3	0.5	0.7	1.0	0.01	0.3	0.5	0.7	1.0
Equiv No Delimit	225	221	199	233	216	159	77	74	77	161	58	68	67	61	142
Equiv Delimit	82	73	564	198	162	50	55	65	121	156	63	74	79	49	128
TypeScript	202	204	185	161	167	85	64	101	107	165	116	107	129	135	173



(a) Equivalence, CodeLlama-7b, all temperatures



(b) Typescript, CodeLlama-7b, all temperatures

Fig. 11. Distribution of how many tokens were proven unrealizably by semantic constrained decoding to produce each individual token. The kth bar gives the number of successful tokens that were produced after trying between $5k$ and $5k + 4$ unsuccessful tokens by CodeLlama-7b.

of the tokens produced during semantic constrained decoding for TypeScript pass on the first try (similarly, 96.1% of tokens for GCD).

7 Related Work

Constrained decoding. Constrained decoding techniques ensure the output of language models meets a given specification by throwing away invalid next tokens at each step. Grammar-constrained decoding, in which the constraint is given as a context-free grammar, has been well studied [14, 36, 41, 42, 45]. A significant portion of ChopChop's runtime is spent pruning tokens that fail simple syntactic validity. Integrating fast syntactic filtering tools such as LLGuidance [30] as a pre-processing step could reduce this cost by eliminating invalid tokens early. Beyond syntax, more recent work has explored enforcing specific semantic constraints such as type safety [31]. Frameworks such as monitors [1] and completion engines [38] provide abstractions for providing more complex constraints by allowing a user to provide a monitor (written in a general-purpose language) that performs the decoding. The main difference between our work and those techniques is that they require the user to write checkers over strings. By contrast, ChopChop operates at the level of abstract syntax, abstracting away the syntactic component and allowing the user to define pruners at the level of program spaces, thus enabling new applications such as equivalence-guided decoding.

Algebraic approaches to parsing. We build on a long line of work viewing parsers and grammars as algebraic, recursive structures [25, 27]. Might et al. [28] presents a functional approach to parsing based on applying Brzozowski derivatives to parser combinators. Zipper-based variants such as [12, 13] reduce redundant traversals in the basic version of parsing with derivatives to

improve efficiency. Integrating these techniques into our implementation could be another avenue to improve performance.

Regular coinduction. Our implementation relies on regular coinduction to represent and manipulate cyclic program spaces. CoCaml [20, 21] is a framework to unambiguously define and compute recursive functions over regular codata. Because some recursive functions admit more than one interpretation on codata, the CoCaml language allows users to define custom solvers to implement their desired semantics. We do not use the CoCaml language directly. However, our Python backend handles the computation of corecursive functions, which produce codata (e.g., our pruners), with a solver analogous to the corec solver presented in [20]. Our backend's solver for `@fixpoint`-annotated functions, which compute concrete values over regular codata, is analogous to the fixpoint solver presented in [20].

Well-behaved corecursion. As discussed in Section 4.1, ChopChop does not prevent users from writing corecursive functions that will not terminate. Finding syntactic conditions for well-behaved corecursive functions is challenging in general. One closely related body of work is on techniques for determining *productivity* of corecursive functions (i.e., corecursive functions that are guaranteed to always eventually produce more of the infinite term they are computing [10]). Famously, a sufficient condition for productivity is *guarded corecursion*, which stipulates that every recursive call to a function must appear under a datatype constructor [10]. This condition is often too restrictive, and a number of variants have been introduced to weaken it [4, 6]. In our case, productivity is necessary (but not sufficient) for regularity or termination of our CoCaml-like backend. For example, `f: int -> Stream` where `f(j) = j::f(j+1)` is a guarded function, and hence productive, but it is not regular and will not terminate in our backend.

Unrealizability and pruning in synthesis. Our approach draws inspiration from the concept of unrealizability—the problem of determining whether no solution exists that satisfies a given specification [18]. Existing approaches to proving unrealizability [18, 19] typically focus on specific synthesis domains, leveraging domain insights to solve particular tasks. Pruning, for example based on types [34] or examples [2], to remove infeasible portions of the search space is a well-established technique in program synthesis. Our work provides a framework to adapt these general methods from traditional synthesis towards constraining the output of LLMs.

Abstract interpretation. A natural question to ask is whether analyses that use abstract interpretation [11] to detect invalid programs can be encoded as pruners in ChopChop. In general, the complexity of the abstract domain affects how one can connect programs to abstract values.

When the set of abstract predicates is *finite*, Wang et al. [43] have shown how to connect programs in a search space with their corresponding abstract values using abstract finite tree automata (AFTA). In an AFTA, each state represents a predicate or abstract value, and the trees derivable from that state correspond exactly to the programs to which the abstract interpretation will assign the state's abstract value. We can encode an AFTA by defining, for each state, a pruning function that encodes the transitions to that state. The example we used in Section 2 involving odd/even predicates is already a (simple) instantiation of such an encoding.

For more complex *infinite* abstract domains (e.g., numerical ones), assigning abstract values to parts of the program space becomes harder. Prior work in synthesis and unrealizability [19] shows that grammar-flow analysis (GFA) [29] provides a systematic solution to do so. GFA analyzes programs in a regular tree grammar (in our case, the program space) by performing dataflow analysis and assigning abstract values from a semilattice to each nonterminal. In a GFA, abstract transformers assigned to individual operations are lifted to grammars by interpreting the unions of productions as joins in the semilattice. By solving the "abstract" equations resulting from these

interpretations, one can obtain a representation of the program space that matches the given abstract interpretation. While solving these equations can be challenging, prior work has identified cases where doing so is feasible [22].

Bidirectional Typing. In a bidirectional typesystem [37], type checking and type inference are done simultaneously. As noted in Section 5.2, this style of typechecking can be naturally expressed as a pruner in ChopChop. Type checking corresponds to describing how to build, via a corecursive function, a well-typed space of programs from well-typed component subspaces. Type inference corresponds to performing an analysis on a space of programs via a fixpoint computation.

8 Conclusion

We introduced ChopChop, a new framework for semantic constrained decoding that allows one to impose semantic constraints directly on the abstract syntax trees representing programs (instead of their string syntax). ChopChop lets users program constraints by providing *semantic pruners*—recursive program operating over finite representations of the infinitely many programs the LM can produce on a given prefix. This flexibility enables new applications—e.g., constraining LMs to only output programs that are equivalent (up to rewrite rules) to a given input program.

ChopChop bridges the gap between formal methods and the outputs of language models. Looking forward, there are numerous opportunities to enhance the framework. In terms of programmability, users of ChopChop must directly define a pruner for the property they wish to enforce as a corecursive function on program spaces. Moreover, this function must return a space that is regular. There may be a better interface for programming pruners that abstracts away these details. In terms of performance, the current implementation of ChopChop is relatively unoptimized, incurring noticeable per-token overhead: future work could likely greatly improve the runtime performance. Exploring new backends for reasoning about codata, such as term rewriting solvers [46, 47] or constrained Horn clauses [16], could enable new applications. Finally, we envision better combinators for expressing semantic pruners and for automatically proving whether pruners over-approximate or under-approximate a user-provided constraint.

Acknowledgments

The project is supported in part by a Microsoft Faculty Fellowship; a UCSD JSOE Scholarship; and NSF under grants CCF-2422214, CCF-2506134, CCF-2446711. This material is based upon work supported by the National Science Foundation Graduate Research Fellowship Program under Grant No. DGE-2038238. Any opinions, findings, and conclusions or recommendations expressed in this publication are those of the authors, and do not necessarily reflect the views of the sponsoring entities. Loris D'Antoni holds concurrent appointments as a Professor at the University of California San Diego and as an Amazon Scholar. This paper describes work performed at the University of California San Diego and is not associated with Amazon.

Data-Availability Statement

The code for this paper is available on GitHub. An artifact containing a Docker image is available on Zenodo [32].

References

[1] Lakshya A Agrawal, Aditya Kanade, Navin Goyal, Shuvendu K. Lahiri, and Sriram K. Rajamani. 2023. Monitor-guided decoding of code LMs with static analysis of repository context. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 1401, 29 pages. doi:10.5555/3666122.3667523

[2] Rajeev Alur, Rastislav Bodik, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. 2013. Syntax-Guided Synthesis. In *Proceedings of the IEEE International Conference on Formal Methods in Computer-Aided Design (FMCAD)*. 1–17. doi:10.1109/FMCAD.2013.6679385

[3] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. arXiv:2108.07732 [cs.PL] https://arxiv.org/abs/2108.07732

[4] Yves Bertot and Ekaterina Komendantskaya. 2009. Using Structural Recursion for Corecursion. In *Types for Proofs and Programs*, Stefano Berardi, Ferruccio Damiani, and Ugo de'Liguoro (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 220–236. doi:10.1007/978-3-642-02444-3_14

[5] Gavin Bierman, Martín Abadi, and Mads Torgersen. 2014. Understanding TypeScript. In *Proceedings of the 28th European Conference on ECOOP 2014 — Object-Oriented Programming - Volume 8586*. Springer-Verlag, Berlin, Heidelberg, 257–281. doi:10.1007/978-3-662-44202-9_11

[6] Jasmin Christian Blanchette, Andrei Popescu, and Dmitriy Traytel. 2015. Foundational extensible corecursion: a proof assistant perspective. *SIGPLAN Not.* 50, 9 (Aug. 2015), 192–204. doi:10.1145/2858949.2784732

[7] Janusz A. Brzozowski. 1964. Derivatives of Regular Expressions. *J. ACM* 11, 4 (Oct. 1964), 481–494. doi:10.1145/321239.321249

[8] Venanzio Capretta. 2005. General Recursion via Coinductive Types. *Logical Methods in Computer Science* 1, 2 (2005). doi:10.2168/LMCS-1(2:4)2005

[9] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. 2022. MultiPL-E: A Scalable and Extensible Approach to Benchmarking Neural Code Generation. arXiv:2208.08227 [cs.LG] https://arxiv.org/abs/2208.08227

[10] Thierry Coquand. 1994. Infinite objects in type theory. In *Types for Proofs and Programs*, Henk Barendregt and Tobias Nipkow (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 62–78. doi:10.1007/3-540-58085-9_72

[11] Patrick Cousot and Radhia Cousot. 1977. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages* (Los Angeles, California) *(POPL '77)*. Association for Computing Machinery, New York, NY, USA, 238–252. doi:10.1145/512950.512973

[12] Pierce Darragh and Michael D. Adams. 2020. Parsing with zippers (functional pearl). *Proc. ACM Program. Lang.* 4, ICFP, Article 108 (Aug. 2020), 28 pages. doi:10.1145/3408990

[13] Romain Edelmann, Jad Hamza, and Viktor Kunčak. 2020. Zippy LL(1) parsing with derivatives. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation* (London, UK) *(PLDI 2020)*. Association for Computing Machinery, New York, NY, USA, 1036–1051. doi:10.1145/3385412.3385992

[14] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. 2023. Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning. In *The 2023 Conference on Empirical Methods in Natural Language Processing*. doi:10.18653/v1/2023.emnlp-main.674

[15] GitHub. 2022. Top Programming Languages - Octoverse 2022. Online. https://octoverse.github.com/2022/top-programming-languages Accessed: 2025-07-07.

[16] Arie Gurfinkel. 2022. Program Verification with Constrained Horn Clauses (Invited Paper). In *Computer Aided Verification*, Sharon Shoham and Yakir Vizel (Eds.). Springer International Publishing, Cham, 19–29. doi:10.1007/978-3-031-13185-1_2

[17] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2020. The Curious Case of Neural Text Degeneration. In *International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=rygGQyrFvH

[18] Qinheping Hu, Jason Breck, John Cyphert, Loris D'Antoni, and Thomas Reps. 2019. Proving Unrealizability for Syntax-Guided Synthesis. In *Computer Aided Verification: 31st International Conference, CAV 2019, New York, NY, USA, July 13-17, 2019, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 11561)*. Springer, Springer, 335–352. doi:10.1007/978-3-030-25540-4_18

[19] Qinheping Hu, John Cyphert, Loris D'Antoni, and Thomas Reps. 2020. Exact and approximate methods for proving unrealizability of syntax-guided synthesis problems. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation* (London, UK) *(PLDI 2020)*. Association for Computing Machinery, New York, NY, USA, 1128–1142. doi:10.1145/3385412.3385979

[20] Jean-Baptiste Jeannin, Dexter Kozen, and Alexandra Silva. 2017. CoCaml: Functional Programming with Regular Coinductive Types. *Fundam. Informaticae* 150, 3-4 (2017), 347–377. doi:10.3233/FI-2017-1473

[21] Jean-Baptiste Jeannin, Dexter Kozen, and Alexandra Silva. 2013. Language Constructs for Non-Well-Founded Computation. In *Programming Languages and Systems*, Matthias Felleisen and Philippa Gardner (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 61–80. doi:10.1007/978-3-642-37036-6_4

[22] Keith J.C. Johnson, Rahul Krishnan, Thomas Reps, and Loris D'Antoni. 2024. Automating Pruning in Top-Down Enumeration for Program Synthesis Problems with Monotonic Semantics. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 304 (Oct. 2024), 27 pages. doi:10.1145/3689744

[23] Jinwoo Kim, Loris D'Antoni, and Thomas Reps. 2023. Unrealizability Logic. *Proc. ACM Program. Lang.* 7, POPL, Article 23 (Jan. 2023), 30 pages. doi:10.1145/3571216

[24] Jinwoo Kim, Shaan Nagy, Thomas Reps, and Loris D'Antoni. 2025. Semantics of Sets of Programs. *Proc. ACM Program. Lang.* 9, OOPSLA1, Article 110 (April 2025), 27 pages. doi:10.1145/3720515

[25] Neelakantan R. Krishnaswami and Jeremy Yallop. 2019. A typed, algebraic approach to parsing. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Phoenix, AZ, USA) *(PLDI 2019)*. Association for Computing Machinery, New York, NY, USA, 379–393. https://doi.org/10.1145/3314221.3314625

[26] Tessa Lau, Steven A. Wolfman, Pedro Domingos, and Daniel S. Weld. 2003. Programming by Demonstration Using Version Space Algebra. *Machine Learning* 53, 1 (2003), 111–156. doi:10.1023/A:1025671410623

[27] Hans Leiß. 1991. Towards Kleene Algebra with Recursion. In *Proceedings of the 5th Workshop on Computer Science Logic (CSL '91)*. Springer-Verlag, Berlin, Heidelberg, 242–256. doi:10.1007/BFb0023771

[28] Matthew Might, David Darais, and Daniel Spiewak. 2011. Parsing with derivatives: a functional pearl. In *Proceedings of the 16th ACM SIGPLAN International Conference on Functional Programming* (Tokyo, Japan) *(ICFP '11)*. Association for Computing Machinery, New York, NY, USA, 189–195. doi:10.1145/2034773.2034801

[29] Ulrich Möncke and Reinhard Wilhelm. 1991. Grammar Flow Analysis. In *Proceedings on Attribute Grammars, Applications and Systems*. Springer-Verlag, Berlin, Heidelberg, 151–186. doi:10.1007/3-540-54572-7_6

[30] Michał Moskal, Hudson Cooper, Aaron Pham, Devise Lucato, Steph Wolski, and Ying Xiong. 2025. guidance-ai/llguidance. https://github.com/guidance-ai/llguidance

[31] Niels Mündler, Jingxuan He, Hao Wang, Koushik Sen, Dawn Song, and Martin Vechev. 2025. Type-Constrained Code Generation with Language Models. *Proc. ACM Program. Lang.* 9, PLDI, Article 171 (June 2025), 26 pages. doi:10.1145/3729274

[32] Shaan Nagy, Timothy Zhou, Loris D'Antoni, and Nadia Polikarpova. 2025. *ChopChop Artifact*. doi:10.5281/zenodo.17337963

[33] Shaan Nagy, Timothy Zhou, Nadia Polikarpova, and Loris D'Antoni. 2025. ChopChop: a Programmable Framework for Semantically Constraining the Output of Language Models. arXiv:2509.00360 [cs.PL] https://arxiv.org/abs/2509.00360

[34] Peter-Michael Osera and Steve Zdancewic. 2015. Type-and-example-directed program synthesis. *SIGPLAN Not.* 50, 6 (June 2015), 619–630. doi:10.1145/2813885.2738007

[35] Kanghee Park, Jiayu Wang, Taylor Berg-Kirkpatrick, Nadia Polikarpova, and Loris D'Antoni. 2025. Grammar-aligned decoding. In *Proceedings of the 38th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) *(NIPS '24)*. Curran Associates Inc., Red Hook, NY, USA, Article 774, 22 pages. doi:10.52202/079017-0774

[36] Kanghee Park, Timothy Zhou, and Loris D'Antoni. 2025. Flexible and Efficient Grammar-Constrained Decoding. arXiv:2502.05111 [cs.CL] https://arxiv.org/abs/2502.05111

[37] Benjamin C. Pierce and David N. Turner. 2000. Local type inference. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 22, 1 (2000), 1–44. doi:10.1145/345099.345100

[38] Gabriel Poesia, Oleksandr Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. 2022. Synchromesh: Reliable code generation from pre-trained language models. arXiv:2201.11227 [cs.LG] https://arxiv.org/abs/2201.11227

[39] Oleksandr Polozov and Sumit Gulwani. 2015. FlashMeta: a framework for inductive program synthesis. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications* (Pittsburgh, PA, USA) *(OOPSLA 2015)*. Association for Computing Machinery, New York, NY, USA, 107–126. doi:10.1145/2814270.2814310

[40] Dan Suciu, Yisu Remy Wang, and Yihong Zhang. 2025. Semantic foundations of equality saturation. *International Conference on Database Theory* (2025). doi:10.4230/LIPIcs.ICDT.2025.11

[41] Shubham Ugare, Tarun Suresh, Hangoo Kang, Sasa Misailovic, and Gagandeep Singh. 2025. SynCode: LLM Generation with Grammar Augmentation. *Transactions on Machine Learning Research* (2025). https://openreview.net/forum?id=HiUZtgAPoH

[42] Bailin Wang, Zi Wang, Xuezhi Wang, Yuan Cao, Rif A. Saurous, and Yoon Kim. 2023. Grammar prompting for domain-specific language generation with large language models. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 2837, 26 pages. doi:10.5555/3666122.3668959

[43] Xinyu Wang, Isil Dillig, and Rishabh Singh. 2017. Program synthesis using abstraction refinement. *Proc. ACM Program. Lang.* 2, POPL, Article 63 (Dec. 2017), 30 pages. doi:10.1145/3158151

[44] J.B. Wells. 1999. Typability and type checking in System F are equivalent and undecidable. *Annals of Pure and Applied Logic* 98, 1 (1999), 111–156. doi:10.1016/S0168-0072(98)00047-5

[45] Brandon T. Willard and Rémi Louf. 2023. Efficient Guided Generation for Large Language Models. arXiv:2307.09702 [cs.CL] https://arxiv.org/abs/2307.09702

[46] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. 2021. egg: Fast and extensible equality saturation. *Proc. ACM Program. Lang.* 5, POPL, Article 23 (Jan. 2021), 29 pages. doi:10.1145/3434304

[47] Yihong Zhang, Yisu Remy Wang, Oliver Flatt, David Cao, Philip Zucker, Eli Rosenthal, Zachary Tatlock, and Max Willsey. 2023. Better Together: Unifying Datalog and Equality Saturation. *Proc. ACM Program. Lang.* 7, PLDI, Article 125 (June 2023), 25 pages. doi:10.1145/3591239

Received 2025-07-10; accepted 2025-11-06